[디버깅] 메모리 누수를 점검하는 방법 - CRTDBG Snapshot(예제 포함)

2020.04.14 15:53 Programming/Debug…

http://hhns.co.kr/ 광고

누수점검 현해환경 AS 1년증명서발급 우수기업

누수점검,보험청구전문업체,원인파악 및 누수탐지후 누수공사로 근본적 해결을 합니다

https://molepipe.com/ 광고

평택시 누수점검 두더지배관! 문의 즉시 친절 전문가 출동

투명한 작업 비용 공개, 누수점검, 빠르게 출동해서 친절하게 해결합니다!

소개

메모리 누수와 관련된 포스팅은 아래 링크를 참고해주세요.

[디버깅] 메모리 누수와 점검하는 방법 소개

[디버깅] 메모리 …

소개 Unmanaged Code인
C++은 메모리 할당과 해제…

luckygg.tistory.com

지난 포스팅을 통해 CRTDBG를 이용한 메모리 누수 점검 방법을 소개했었습니다.

[디버깅] 메모리 누수를 점검하는 방법 - CRTDBG (예제 포함)

[디버깅] 메모리 …

메모리 누수와 관련된
포스팅은 아래 링크를…

luckygg.tistory.com

그런데 CRT 라이브러리를 이용한 다른 한 가지 방법이 더 있습니다. 이번 포스팅을 통해 CRT 라이브러리의 스냅숏을 이용한 메모리 누수 점검 방법을 소개하겠습니다.

CRT 스냅숏

CRT 라이브러리에는 `_CrtMemState`라는 구조체가 정의되어 있습니다. 이 구조체는 디버그 힙(Debug Heap)의 첫 번째 block 정보를 가지고 있습니다. 그리고 이 구조체를 이용한 `_CrtMemCheckpoint` 함수가 있습니다. 이는 호출된 순간에 디버그 힙의 현재 상태에 대한 스냅숏을 만들게 됩니다. 이 정보는 `_CrtMemState`에 저장됩니다. 생성된 스냅숏은 메모리 누수를 비롯한 여러 정보가 있으며 `_CrtMemDifference`로 두 스냅숏의 차이점을 반환하여 메모리 상태를 점검할 수 있습니다. 물론, 이런 점검은 디버그 모드 일 때에만 가능합니다.

C++ Console 예시

직접 예제를 만들어 테스트해 보겠습니다. 아래 코드처럼 작성합니다.

```
#include "pch.h"
#include <Windows.h>
#include <iostream>
#define _CRTDBG_MAP_ALLOC
#include <cstdlib>
#include <crtdbg.h>
#ifdef _DEBUG
#define new new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )
#endif
int main()
{
_CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF);
_CrtMemState s1;
_CrtMemCheckpoint(&s1);
_CrtMemDumpStatistics(&s1);
byte* ptr = new byte[100];
//delete[] ptr;
```

```
_CrtMemState s2;
_CrtMemCheckpoint(&s2);
_CrtMemDumpStatistics(&s2);
_CrtMemState s3;
if (_CrtMemDifference(&s3, &s1, &s2) == true)
_CrtMemDumpStatistics(&s3);
}
```

`_CrtMemState`는 스냅숏 정보를 갖는 구조체입니다. 메모리 할당 전인 s1, 메모리 할당 후인 s2 그리고 `s1`과 `s2`의 차이점 정보를 갖는 `s3`을 선언합니다.

`_CrtMemCheckpoint`는 스냅숏 정보를 생성하는 함수입니다. 메모리 할당 전, 후에 스냅숏을 생성합니다.

`_CrtMemDifference`는 두 스냅숏의 정보를 비교하고, 차이가 있으면 차이점 정보를 생성합니다.

`_CrtMemDumpStatistics`는 스냅숏 정보 구조체인 `_CrtMemState`를 인자로 받아 출력 창에 정보를 표시합니다.

디버그 모드로 실행 후 종료하면, 아래의 메시지가 출력됩니다.

```
// 메모리 할당 전 메시지
0 bytes in 0 Free Blocks.
0 bytes in 0 Normal Blocks.
19122 bytes in 113 CRT Blocks.
0 bytes in 0 Ignore Blocks.
0 bytes in 0 Client Blocks.
Largest number used: 19122 bytes.
Total allocations: 44955 bytes.

// 메모리 할당 후 메시지
0 bytes in 0 Free Blocks.
100 bytes in 1 Normal Blocks.
19122 bytes in 113 CRT Blocks.
0 bytes in 0 Ignore Blocks.
0 bytes in 0 Client Blocks.
Largest number used: 19222 bytes.
Total allocations: 45055 bytes.

// 두 스냅숏의 차이 메시지
0 bytes in 0 Free Blocks.
100 bytes in 1 Normal Blocks.
0 bytes in 0 CRT Blocks.
0 bytes in 0 Ignore Blocks.
```

```
0 bytes in 0 Client Blocks.
Largest number used: 100 bytes.
Total allocations: 100 bytes.

Detected memory leaks!
Dumping objects ->
c:₩users₩gg₩desktop₩consoleapplication1₩consoleapplication1₩consoleapplication1.cpp(22) : {164} normal block at 0x000A6010, 100 bytes long.
 Data: <                > CD CD CD CD CD CD CD CD CD CD CD CD CD CD CD CD
Object dump complete.
```

차이점 정보를 보면 `Total allocations: 100 bytes.`가 표시되어 있습니다. 정확하게 정보를 표시합니다.

만약, 올바르게 `delete[] ptr;`을 정상적으로 호출하면 결과가 어떨까요?

```
0 bytes in 0 Free Blocks.
0 bytes in 0 Normal Blocks.
15026 bytes in 112 CRT Blocks.
0 bytes in 0 Ignore Blocks.
0 bytes in 0 Client Blocks.
Largest number used: 16530 bytes.
Total allocations: 40859 bytes.

// 메모리 할당과 해제 후
0 bytes in 0 Free Blocks.
0 bytes in 0 Normal Blocks.
15026 bytes in 112 CRT Blocks.
0 bytes in 0 Ignore Blocks.
0 bytes in 0 Client Blocks.
Largest number used: 16530 bytes.
Total allocations: 40959 bytes.
```

위 결과를 보면 알 수 있듯이, 두 스냅숏의 정보가 동일하여 따로 출력된 메시지가 없습니다. 그리고 메모리 누수 메시지도 보이지 않습니다.